S01

Getting set up: R, RStudio and QGIS

R PROGRAMMING GIS & MAPPING

Today you install the tools for the whole course, learn how R thinks about data, and open the same survey dataset in both R and QGIS. You also learn to set up the RStudio project you will create at the start of every session.

DURATION	About 2 hours, with a short break.
YOU NEED	A laptop with R, RStudio Desktop and QGIS (Long Term Release). Admin rights help.
FILES	`session01_intro_to_R.R`, `session01_solutions.R`, `survey_points.csv`
DATA	40 bird point-count stations in the Sierra de Montfred, a fictitious natural park, from 145 to 1,680 m. The data are simulated for teaching.

Learning outcomes

By the end of this session you can:

- set up an RStudio project with a clear folder structure;
- create objects, vectors and data frames, and use functions such as `mean()` and `table()`;
- read a CSV file into R, inspect it and select rows that meet a condition;
- make a histogram, a boxplot and a first point map in base R;
- load the same coordinates in QGIS, style them by habitat and save a GeoPackage.

1 · Install the software

Install in this order. R is the language; RStudio is the editor you use to write and run R; QGIS is a free desktop GIS. Accept the default options unless stated otherwise.

Software	Where to get it	Notes
R	cran.r-project.org	Choose your operating system. On a Mac, pick the build for Apple silicon (M1 or newer) or Intel.
RStudio Desktop	posit.co/download/rstudio-desktop	The free version. Install after R.
QGIS	qgis.org/download	Choose the **Long Term Release (LTR)**: the most stable version for teaching.
ArcGIS Pro (optional)	Through your university licence	Windows only. We use it in Session 7; QGIS covers everything else.

If something goes wrong

RStudio says it cannot find R: install R first, then restart RStudio.
Windows blocks the installer: right-click it and choose *Run as administrator*, or ask IT.
Packages fail to install: on Windows, also install *Rtools* from the same CRAN page. On a Mac, run `xcode-select --install` in the Terminal once.

2 · RStudio and your course project

RStudio has four panes. You will mostly write in the **Source** pane (top left) and see results in the **Console** (bottom left). The **Environment** pane (top right) lists the objects you have created, and the bottom-right pane shows files, plots and help.

Create the session's project

1. In RStudio choose **File > New Project > New Directory > New Project**.
2. Name it `session01` and keep it inside a course folder, for example Documents/Spatial_Ecology. Click **Create Project**.
3. In the Files pane, click **New Folder** three times to create `1.Data`, `2.Scripts` and `3.Outputs`.
4. Copy `survey_points.csv` into `1.Data/` and the scripts into `2.Scripts/`.
5. To return to this project, double-click `session01.Rproj`. R then starts in the right folder, so short paths such as `"1.Data/survey_points.csv"` work on every computer.

```
Spatial_Ecology/
|-- session01/
|   |-- session01.Rproj
|   |-- 1.Data/        raw data: never edit these files by hand
|   |-- 2.Scripts/     the session's script and solutions
|   `-- 3.Outputs/     tables and figures your scripts create
|-- session02/         same structure, new project
`-- ...
```

A new project every session

At the start of every session you will create a new project the same way (session02, session03…), with the same three folders, and save that day's data in 1.Data. Each project then holds everything needed to rerun that session's analysis, and a mistake in one project does not affect the others.

Two settings that save trouble

Go to **Tools > Global Options > General**. Untick *Restore .RData into workspace at startup* and set *Save workspace to .RData on exit* to **Never**. Your script, not a hidden workspace file, should be the record of your analysis, so anyone (including you in six months) can rerun it from scratch.

3 · First steps in R

Open `session01_intro_to_R.R` and run it one line at a time with **Ctrl + Enter** (**Cmd + Enter** on a Mac). These are the ideas to take away.

Objects and vectors

The arrow `<-` stores a value under a name. A vector holds several values of the same type, and R applies most operations to every value at once.

```
robins <- c(4, 2, 0, 5, 3, 1)     # robins counted at six points
mean(robins)                       # 2.5
robins[robins > 2]                 # 4 5 3
counts <- c(3, NA, 5, 2)           # NA = not recorded
mean(counts, na.rm = TRUE)         # 3.33
```

NA is not zero

In field data, 0 means "we looked and saw none". NA means "no value was recorded". Treating one as the other changes your results. We meet this again in Session 2.

Data frames

A data frame is a table: one row per observation, one column per variable. The survey file has one row per point.

```
points <- read.csv("1.Data/survey_points.csv")
str(points)            # column names and types
table(points$habitat)
```

Column	Meaning
`point_id`	Survey point code, P01 to P40
`longitude, latitude`	Position in decimal degrees (WGS 84, EPSG:4326)
`elevation_m`	Elevation above sea level, metres
`habitat`	Dominant habitat within 100 m of the point
`canopy_cover_pct`	Tree canopy cover, %
`dist_stream_m`	Distance to the nearest stream, metres

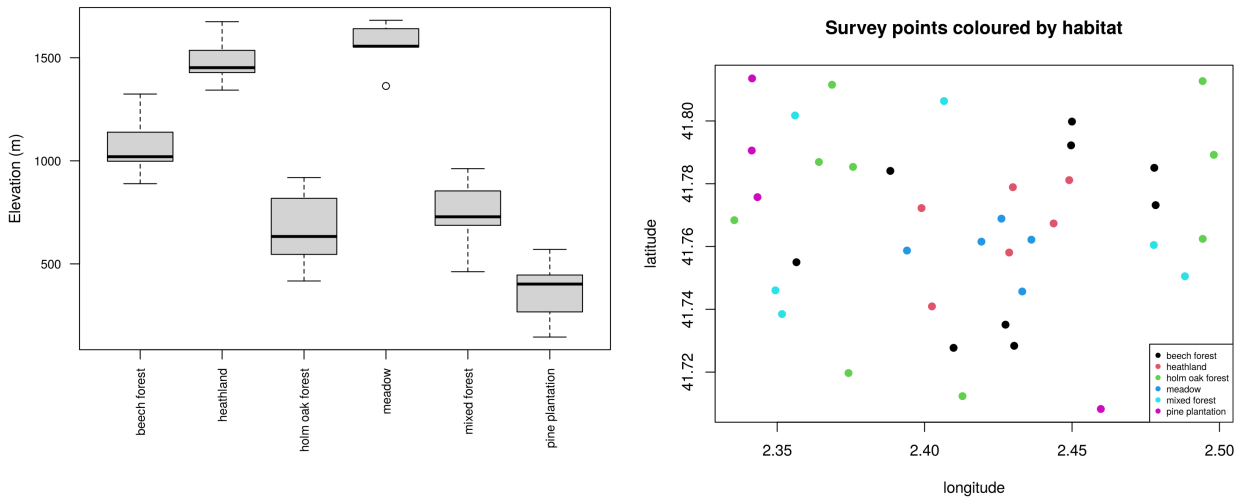
`table(points$habitat)` shows how the 40 points are spread across habitats:

beech forest	heathland	holm oak forest	meadow	mixed forest	pine plantation
9	6	9	5	6	5

Selecting rows and summarising groups

```
beech <- points[points$habitat == "beech forest", ]   # == tests equality
high  <- points[points$elevation_m > 1200, ]
aggregate(elevation_m ~ habitat, data = points, FUN = mean)
```

The group means already tell an ecological story: habitats are arranged along the elevation gradient, from pine plantations (about 370 m) and holm oak and mixed forest (650–740 m), through beech forest (about 1,070 m), to summit heathland and meadows above 1,450 m.



Left: elevation of survey points by habitat. Right: a first map, with longitude on the x-axis and latitude on the y-axis. It is not yet a proper map, because degrees of longitude and latitude are not the same length; we fix this in Sessions 5 and 6.

Key functions from today

Function	What it does
`c()`	Combine values into a vector
`mean(), sum(), range(), length()`	Basic summaries of a vector
`table()`	Count how often each category occurs
`read.csv(), write.csv()`	Read and write CSV files
`head(), str(), summary(), nrow()`	Inspect a data frame
`df[rows, cols]`	Select rows and columns; rows can be a condition
`aggregate()`	Summarise a variable by group
`hist(), boxplot(), plot()`	Base R plots
`?name`	Open the help page of a function

4 · The same points in QGIS

A CSV with coordinate columns can be read straight into a GIS. Seeing the data on a basemap is the quickest way to spot errors, such as a point in the sea or swapped latitude and longitude. The points sit at real coordinates but the landscape is simulated, so it will not match the basemap.

1. Open QGIS and start a new project with **Project > New**.
2. Choose **Layer > Add Layer > Add Delimited Text Layer** (Ctrl + Shift + T).
3. Set **File name** to `1.Data/survey_points.csv` and **File format** to CSV.
4. Under **Geometry Definition** choose *Point coordinates*, set **X field** = `longitude`, **Y field** = `latitude` and **Geometry CRS** = `EPSG:4326 - WGS 84`. Click **Add**, then **Close**.
5. Add a basemap: in the **Browser** panel expand **XYZ Tiles** and double-click **OpenStreetMap**. In the **Layers** panel, drag it below the points.
6. Right-click the points layer > **Properties > Symbology**. Change *Single Symbol* to **Categorized**, set **Value** to `habitat`, click **Classify**, then **OK**.

7. Use the **Identify Features** tool (Ctrl + Shift + I) and click a point to see its attributes. Press **F6** to open the attribute table.
8. Save a proper spatial file: right-click the layer > **Export > Save Features As**. Format **GeoPackage**, file name `1.Data/survey_points.gpkg`. Click **OK**.
9. Save the project with **Project > Save As** as `session01.qgz` in your course folder.

> **Why save a GeoPackage?**
> A delimited text layer stays linked to the CSV and is re-read every time. A GeoPackage (.gpkg) stores geometry, attributes and the coordinate reference system in one open file that both QGIS and R (package `sf`) read directly. It is the format we use from Session 5 onwards, instead of shapefiles that come as several separate files.

In ArcGIS Pro the equivalent is **Map > Add Data > XY Point Data**, which opens the *XY Table To Point* tool. Use the same X and Y fields and the WGS 1984 coordinate system.

Exercises

These are at the end of the script. Try them before opening `session01_solutions.R`.

1. A 2.5 km transect was walked in 50 minutes. Store both values in objects and calculate walking speed in km/h.
2. Blackbirds at eight points: 1, 0, 3, 2, 2, 4, 0, 1. What is the mean, and how many points had none? *Hint:* `sum()` *of a TRUE/FALSE vector counts the TRUEs.*
3. How many survey points are in pine plantation?
4. What are the lowest and highest elevations of the beech forest points?
5. Which point is closest to a stream? Give its `point_id` and habitat. *Hint:* `which.min()`.
6. Plot canopy cover (y) against elevation (x) with a title and axis labels with units. What pattern do you see?
7. **QGIS:** label each point with its `point_id` (layer Properties > **Labels** > *Single Labels*). Then use **Select Features by Expression** with `"elevation_m" > 1200`. How many are selected?

Before Session 2

- Run `install.packages("tidyverse")` in the RStudio Console. It takes a few minutes. If it ends with an error, copy the last lines and send them to me before class.
- Finish the exercises and check your answers against the solutions script.
- Optional reading: chapters 1 to 3 of *R for Data Science* (r4ds.hadley.nz).